OPERATING SYSTEM

COMPLETE DETAILED GTU NOTES

All 9 Units | Long & Detailed Answers | Algorithms | Diagrams | Solved Examples

B.E. Computer / IT Engineering | GTU Exam Preparation

UNIT 1: INTRODUCTION TO OPERATING SYSTEMS (4 Hours)

1.1 Goals and Structure of an Operating System

> Definition: An Operating System (OS) is a system software program that acts as an intermediary (middleman) between the computer user and the computer hardware. It is the most fundamental software installed on a computer and controls all other programs running on the machine.

Without an OS, a user would have to interact directly with the hardware — writing binary instructions to the processor, managing RAM manually, and controlling I/O devices. The OS abstracts this complexity and provides a clean, user-friendly interface. Think of the OS as the manager of a company: it allocates resources (CPU, RAM, disk) to employees (programs/processes) and ensures everyone works efficiently without conflict.

Goals of an Operating System:

1. Convenience (User Goal):

The primary goal of any OS is to make the computer easy to use. Without an OS, users would need to write complex machine language programs just to display something on screen. The OS provides abstractions like files, directories, and windows so users can interact with the system naturally. For example, a user simply double-clicks a file to open it — the OS handles all the underlying disk read operations, memory allocation, and program loading automatically.

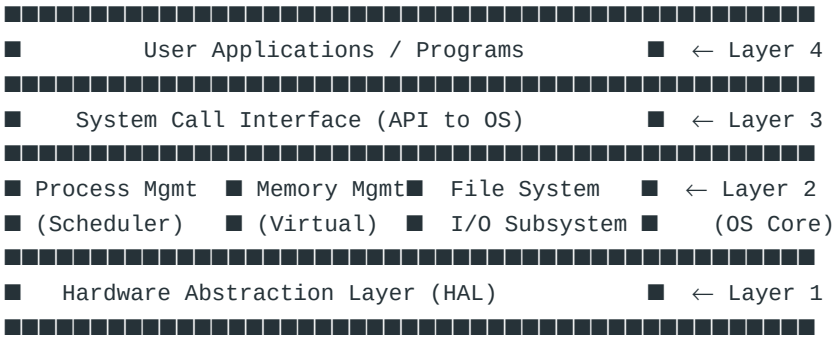
2. Efficiency (System Goal):

Computer hardware (CPU, RAM, I/O devices) is expensive and limited. The OS must ensure these resources are used efficiently with minimal waste. For example, when a process is waiting for an I/O operation (like reading from disk), the OS switches the CPU to another ready process instead of letting it sit idle. This is called multiprogramming, and it dramatically increases CPU utilization (from ~10% in simple batch systems to ~90%+ in modern OS).

3. Ability to Evolve:

An OS must be designed so that new hardware and software features can be added without breaking existing functionality. Modern OS are modular — new device drivers, file systems, or scheduling algorithms can be added as loadable kernel modules without rebooting or reinstalling the OS. Linux's monolithic kernel with loadable modules is a great example of this design.

Structure of Operating System (Layered View):





1.2 Basic Functions of an Operating System

The OS performs the following seven major functions. Each function is a specialized subsystem within the OS kernel:

1. Process Management:

A process is a program in execution. The OS is responsible for: creating and deleting processes, suspending and resuming processes, providing mechanisms for process synchronization and communication, and handling deadlocks. The process manager uses a data structure called the Process Control Block (PCB) to track each process. The CPU scheduler decides which process runs next based on scheduling algorithms like Round Robin or Priority Scheduling.

2. Memory Management:

The OS tracks every byte of main memory — which parts are in use, by which process, and which are free. When a process requests memory, the OS allocates it; when the process terminates, the OS reclaims it. Modern OS use virtual memory, which allows a process to use more memory than physically available by storing inactive pages on disk and loading them on demand (demand paging).

3. File System Management:

The OS provides a uniform interface to store and retrieve data on disk through the file system. It manages: creation, deletion, reading, writing, and searching of files and directories; access control (who can read/write which file); and mapping of file names to physical disk locations. Examples of file systems: ext4 (Linux), NTFS (Windows), APFS (macOS).

4. I/O Device Management:

The OS manages all I/O devices (keyboard, mouse, disk, printer, network card) through device drivers — specialized software modules that communicate with specific hardware. The OS provides a uniform interface so programs can use any device with the same read()/write() calls without knowing hardware details. I/O scheduling (like disk scheduling) also falls under this function.

5. Security and Protection:

The OS enforces access controls to protect processes from each other and protect system resources from unauthorized access. Protection mechanisms include: user authentication (login), permission bits (read/write/execute), memory protection (processes cannot access each other's memory), and system call filtering. In Linux, each file has owner, group, and other permissions (rwxrwxrwx format).

6. Networking:

Modern OS include a network subsystem (TCP/IP stack) that manages network connections, data transmission, and protocols. The OS provides socket API so programs can communicate over a network with simple read/write operations, abstracting the complex underlying protocols.

7. Error Detection and Correction:

The OS must constantly monitor the computer for errors at all levels: hardware errors (bad memory sector, CPU parity error), software errors (illegal instruction, division by zero), and I/O errors (disk read error). When an error is detected, the OS must respond appropriately — either correcting the error, reporting it to the user, or

terminating the offending process.

1.3 Interaction of OS with Hardware Architecture

The OS and hardware are tightly coupled. The OS is designed around specific hardware features that support protected, efficient operation. The key hardware-OS interactions are:

1. Interrupts:

An interrupt is a signal sent by hardware to the CPU indicating that an event needs immediate attention. When an interrupt occurs: (1) the CPU finishes the current instruction, (2) saves its state (registers, PC) onto the stack, (3) looks up the Interrupt Vector Table to find the address of the Interrupt Service Routine (ISR), (4) jumps to the ISR and handles the interrupt, (5) restores the saved state and resumes the interrupted process. Examples: timer interrupt (for preemptive scheduling), keyboard interrupt (key pressed), disk interrupt (I/O complete), division-by-zero exception.

2. Dual Mode Operation:

To protect the OS from user programs, the CPU operates in two modes, controlled by a mode bit in the processor status register:

- **User Mode (mode bit = 1):** Restricted mode. User programs run here. Cannot execute privileged instructions (like disabling interrupts, accessing I/O ports directly). If a user program tries to execute a privileged instruction, the CPU generates a trap (protection fault) and transfers control to the OS.
- **Kernel Mode (mode bit = 0):** Privileged mode. The OS runs here. Has complete access to all hardware and can execute any instruction. The OS switches to kernel mode only when needed (system call, interrupt, exception) and returns to user mode as quickly as possible.

3. System Calls:

System calls are the programmatic interface between user programs and the OS kernel. When a program needs an OS service (read a file, create a process, allocate memory), it executes a special software interrupt instruction (trap/syscall) which switches the CPU from user mode to kernel mode, passes control to the OS at a predefined entry point, the OS performs the service, then returns to user mode with the result.

4. Direct Memory Access (DMA):

For I/O-intensive operations (like disk read/write), it would be very slow if the CPU had to copy each byte one-by-one between I/O device and memory. DMA solves this: the CPU programs the DMA controller with the source, destination, and count of bytes to transfer. The DMA controller then performs the transfer independently while the CPU continues executing other instructions. When done, the DMA controller sends an interrupt to the CPU. This allows true overlap of CPU computation and I/O transfer.

1.4 Types of Operating Systems — Detailed Explanation

1. Batch Operating System:

In the earliest computers, there was no interactive user interface. Users submitted jobs (programs) as decks of punch cards to an operator. The operator would group similar jobs into batches and load them one at a time. The OS would execute each job sequentially without user interaction. Problems: if a job had an error, it had to wait for the next batch run (which could take hours or days). No interaction between user and program during execution. CPU was idle whenever a job did I/O. Example: IBM OS/360 in the 1960s.

2. Multiprogramming OS:

The key insight: a CPU is idle whenever a process waits for I/O. Multiprogramming solves this by keeping multiple programs in memory simultaneously. When the running process waits for I/O, the OS switches the CPU to another ready process. This keeps the CPU busy almost all the time. The OS must manage memory partitioning among multiple programs and schedule CPU time. This is the foundation of all modern OS.

```
CPU Utilization formula:
  If a process spends fraction 'p' of time waiting for I/O:
  CPU utilization = 1 - p^n  (where n = number of programs in memory)

  Example: p=0.8 (80% I/O wait), n=1 program: utilization = 1-0.8 = 20%
           n=5 programs: utilization = 1 - 0.8^5 = 1 - 0.33 = 67%
           n=10 programs: utilization = 1 - 0.8^10 = 1 - 0.11 = 89%
```

3. Multitasking / Time-Sharing OS:

Multitasking extends multiprogramming by rapidly switching the CPU between processes so frequently that each user gets the impression their process is running continuously. Each process gets a small time slice (quantum, typically 10-100 ms). When the quantum expires, a timer interrupt fires, and the OS switches to the next process (preemption). The user experiences smooth, responsive interaction even though the CPU is shared. Example: Modern Linux, Windows, macOS are all multitasking OS.

4. Real-Time OS (RTOS):

A Real-Time OS guarantees that a process will complete within a specific deadline. This is critical in safety-critical systems where a late response is as bad as no response. Two types:

- **Hard RTOS:** Missing a deadline is a catastrophic failure. Used in: aircraft flight control, nuclear reactor control, medical devices (pacemakers, insulin pumps), anti-lock braking systems. VxWorks, FreeRTOS.
- **Soft RTOS:** Missing a deadline degrades performance but is not catastrophic. Used in: video streaming, online gaming, multimedia. Linux with real-time patches.

5. Distributed OS:

A Distributed OS manages a collection of geographically distributed computers connected via a network, making them appear as a single unified system to the user. The user doesn't know which physical machine is executing their program. Key challenges: network latency, partial failures, consistency of shared data, security. Examples: LOCUS, Amoeba, Google's Borg (manages Google's server farms).

1.5 System Calls — Detailed Explanation

A system call is the mechanism by which a user-level program requests a service from the operating system kernel. It is the controlled entry point into the kernel. System calls are necessary because user programs cannot directly access hardware or kernel data structures — this would violate the protection boundary.

How a System Call Works (Step-by-Step):

```
Step 1: User program calls a library function (e.g., printf, fopen)
Step 2: Library function sets up system call number and arguments in registers
Step 3: Library executes 'TRAP' or 'SYSCALL' instruction
Step 4: CPU switches from User Mode → Kernel Mode
```

```
Step 5: CPU looks up system call number in System Call Table
Step 6: OS kernel executes the requested service
Step 7: Kernel places return value in register, switches back to User Mode
Step 8: Library function returns result to user program

Example: read(fd, buffer, count) in Linux
  → sets EAX=0 (sys_read), EBX=fd, ECX=buffer, EDX=count
  → executes 'int 0x80' (trap)
  → kernel reads data from file, copies to buffer
  → returns number of bytes read in EAX
```

Types of System Calls:

Category	System Calls	Description
Process Control	fork(), exec(), exit(), wait(), getpid()	Create, run, terminate, and manage processes
File Management	open(), close(), read(), write(), lseek(), stat()	Create, delete, read, write, seek files
Device Management	ioctl(), read(), write(), open(), close()	Request/release devices, read/write device data
Information	getpid(), getuid(), time(), uname()	Get/set system time, process ID, system info
Communication	pipe(), socket(), connect(), send(), recv()	Create pipes, sockets, network connections
Memory	mmap(), brk(), sbrk()	Map files into memory, change heap size

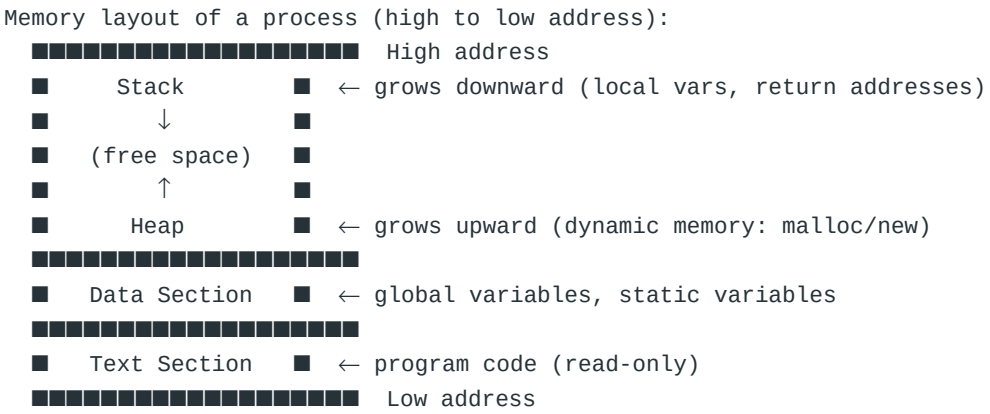
UNIT 2: PROCESS AND THREADS MANAGEMENT (5 Hours)

2.1 Process Concept — Detailed

> Definition: A Process is a program in execution. It is the unit of work in a modern OS. A program is a passive entity — a file sitting on disk. A process is an active entity — a program that has been loaded into memory and is being executed by the CPU.

A process consists of the following components:

- **Text Section (Code Segment):** The compiled machine code of the program. Read-only. Shared among multiple instances of the same program.
- **Data Section:** Contains global and static variables. Initialized data (e.g., int x=5;) and uninitialized data (BSS segment, e.g., int y;).
- **Stack:** Contains temporary data — function parameters, local variables, return addresses. Grows downward in memory. Managed automatically by the CPU (push/pop instructions).
- **Heap:** Dynamically allocated memory (malloc/new). Grows upward. Managed by the programmer (must be freed to avoid memory leaks).
- **Program Counter (PC):** Points to the next instruction to be executed.
- **CPU Registers:** Accumulator, index registers, stack pointer — current working data of the process.

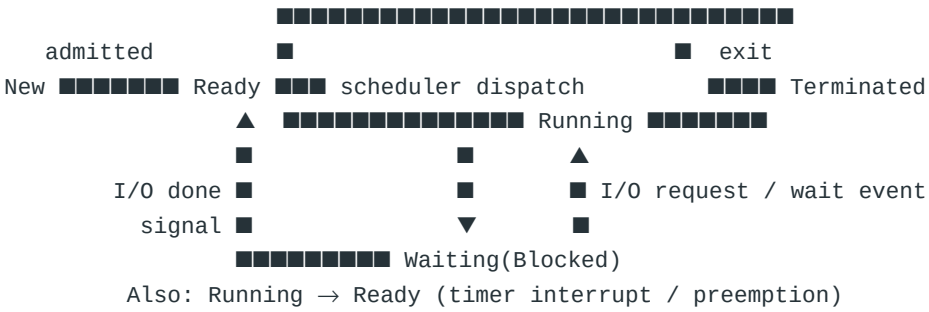


Process State Diagram — 5 State Model (Detailed):

At any point in time, each process is in exactly one of the following five states:

- **New:** The process is being created. OS is allocating PCB, memory, and setting up resources. The process has not yet been admitted to the ready queue.
- **Ready:** The process is in memory, ready to run, waiting for the CPU. Multiple processes can be in the ready state simultaneously. They are kept in a ready queue.
- **Running:** The process currently has the CPU and is executing instructions. Only one process per CPU core can be running at a time.
- **Waiting (Blocked):** The process is waiting for some event (I/O completion, signal, resource). It cannot use the CPU even if it's free. Goes back to Ready when the event occurs.

- **Terminated:** The process has finished execution (normal exit or killed). OS reclaims all its resources (memory, file handles, etc.).



2.2 Process Control Block (PCB) — Detailed

The PCB (also called Task Control Block) is the most important data structure in the OS. Every process that exists in the system has exactly one PCB. The OS maintains a table of PCBs — one for each process. The PCB is the identity card of a process. Without a PCB, a process cannot exist.

The PCB stores all the information needed to manage, suspend, resume, and terminate a process:

Field	Description	Example Value
Process ID (PID)	Unique numeric identifier for the process	1234
Process State	Current state of the process	Running, Ready, Blocked
Program Counter	Address of next instruction to execute	0x40058A
CPU Registers	All register values (saved on context switch)	EAX=5, EBX=12, ESP=0xFF20
Scheduling Info	Priority, scheduling queue pointers	Priority=10, nice=-5
Memory Info	Base/limit registers, page table pointer	Base=0x8000, PageTable=0x3000
I/O Status	List of open files, devices being used	fd[0]=stdin, fd[1]=stdout, fd[3]=myfile.txt
Accounting Info	CPU time used, real time, process limits	CPU_time=2.3s, wall_time=10.1s
Parent PID (PPID)	PID of the parent process	1200
Open File Table	Pointers to open file structures	Files: /etc/passwd, /tmp/data

Context Switch:

A context switch occurs when the OS switches the CPU from one process to another. The steps are: (1) Save the running process's state into its PCB (all registers, PC, stack pointer). (2) Update the process state to Ready or Blocked. (3) Load the new process's state from its PCB. (4) Update the memory management registers (page table pointer). (5) Set the new process to Running state. Context switching is pure overhead — no useful work is done during the switch. Modern CPUs have hardware support to speed up context switches.

2.3 Process vs Thread — Detailed Comparison

A thread is a lightweight unit of execution within a process. A process can contain multiple threads, all sharing the same address space, code, data, and open files. Each thread has its own stack, registers, and program counter.

Real-world analogy: A process is like a company. Threads are like employees in that company. They all share the same office (address space) and resources (files, printers), but each does their own work independently.

Feature	Process	Thread
Definition	Program in execution; independent unit	Lightweight unit within a process
Address Space	Own separate address space	Shares address space of parent process
Memory	Own code, data, heap, stack	Own stack and registers only; shares code, data, heap
Creation Time	Slow (fork() copies entire memory)	Fast (only stack and registers allocated)
Context Switch	Slow (different address space = TLB flush)	Fast (same address space, no TLB flush)
Communication	IPC: pipes, sockets, shared memory (complex)	Shared memory directly (simple but needs sync)
Isolation	Crash in one process does not affect others	Crash in one thread can kill entire process
Resource Sharing	Does not share resources by default	Shares code, data, heap, open files, signals
Scheduling	Scheduled by OS	Scheduled by OS (kernel threads) or library (user threads)
Example	Chrome: each tab is a separate process	Web server: one thread per HTTP request

2.4 CPU Scheduling — Detailed

CPU scheduling is the process of deciding which process in the ready queue gets the CPU next. Since the CPU is the most valuable resource, good scheduling is critical for system performance.

Scheduling Criteria (Metrics):

- **CPU Utilization:** Percentage of time CPU is busy (not idle). Goal: keep as close to 100% as possible. Real systems: 40% (lightly loaded) to 90%+ (heavily loaded).
- **Throughput:** Number of processes completed per unit time. For batch systems: maximize. For interactive: not primary concern.
- **Turnaround Time (TAT):** Total time from process submission to completion. TAT = Completion Time - Arrival Time. Includes waiting in ready queue + CPU execution + I/O wait.
- **Waiting Time (WT):** Total time spent in the ready queue (just waiting, not executing). WT = TAT - Burst Time.
- **Response Time:** Time from submission of request to first response produced. Critical for interactive systems. Not the time to complete the request.

Preemptive vs Non-Preemptive Scheduling:

- **Non-Preemptive:** Once a process gets the CPU, it keeps it until it voluntarily gives up (terminates or waits for I/O). Simpler to implement, no preemption-related bugs, but poor for interactive/real-time use. Examples: FCFS, non-preemptive SJF.
- **Preemptive:** OS can forcibly take the CPU from a running process (timer interrupt). Better response time, fairness, and real-time support, but more complex (need synchronization). Examples: Round Robin, preemptive Priority, SRTF.

2.5 Scheduling Algorithms — Detailed with Full Examples

A) FCFS — First Come First Served

The simplest scheduling algorithm. Processes are served in the order they arrive in the ready queue. Implemented using a FIFO queue. Non-preemptive.

Detailed Example: Processes: P1(AT=0, BT=24), P2(AT=1, BT=3), P3(AT=2, BT=3)

```
Gantt Chart:
| P1                    | P2  | P3  |
0                       24    27    30

Calculations:
Process  AT  BT  CT   TAT(CT-AT)  WT(TAT-BT)
P1        0  24  24      24           0
P2        1   3  27      26          23
P3        2   3  30      28          25

Avg TAT = (24 + 26 + 28) / 3 = 26 ms
Avg WT  = (0 + 23 + 25) / 3 = 16 ms

Problem: Convoy Effect — P2 and P3 (short jobs) must wait a very long time
behind P1 (long job). Causes high average waiting time.
```

B) SJF — Shortest Job First

The process with the shortest CPU burst time is selected next. Provably optimal for minimizing average waiting time among non-preemptive algorithms. The preemptive version is called SRTF (Shortest Remaining Time First).

Non-Preemptive SJF Example: P1(AT=0,BT=7), P2(AT=2,BT=4), P3(AT=4,BT=1), P4(AT=5,BT=4)

```
At t=0: Only P1 available → run P1
At t=7: P2(BT=4), P3(BT=1), P4(BT=4) available → shortest is P3
At t=8: P2(BT=4), P4(BT=4) → tie, pick P2 (FCFS)
At t=12: P4(BT=4) → run P4

Gantt Chart:
| P1      | P3 | P2    | P4    |
0         7    8      12      16

Process  AT  BT  CT   TAT   WT
P1        0   7   7     7    0
P2        2   4  12    10    6
P3        4   1   8     4    3
P4        5   4  16    11    7

Avg WT = (0 + 6 + 3 + 7) / 4 = 4 ms  ← much better than FCFS

Drawback: Cannot know burst time in advance (must estimate from history).
Starvation: Long processes may never run if short ones keep arriving.
```

C) Round Robin (RR) — Time Quantum Based

Round Robin is designed specifically for time-sharing systems. Each process gets a fixed time quantum (q). After q expires, the process is preempted and moved to the back of the ready queue. This ensures every process gets equal CPU time and good response time.

Detailed Example: P1(BT=10), P2(BT=4), P3(BT=5), P4(BT=3). All arrive at t=0. Quantum q=3

```
Gantt Chart:
| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P1 |
0    3    6    9   12   15   16  19   22

Step-by-step execution:
t=0-3:   P1 runs (remaining: 10-3=7)
t=3-6:   P2 runs (remaining: 4-3=1)
t=6-9:   P3 runs (remaining: 5-3=2)
t=9-12:  P4 runs (remaining: 3-3=0) → P4 DONE at t=12
t=12-15: P1 runs (remaining: 7-3=4)
t=15-16: P2 runs (remaining: 1-1=0) → P2 DONE at t=16
t=16-19: P3 runs (remaining: 2-2... wait, 2 < 3, runs for 2) → DONE at t=18
t=18-22: P1 runs last 4 ms → DONE at t=22

Process  BT  CT   WT
P1       10  22   12
P2        4  16    9
P3        5  18    8
P4        3  12    6
Avg WT = (12+9+8+6)/4 = 8.75 ms

Key insight: Small q → better response time but more context switches.
Large q → degenerates to FCFS. Typical q = 10-100 ms.
```

D) Priority Scheduling

Each process is assigned a priority number. The CPU is allocated to the process with the highest priority (lowest number = highest priority in many implementations, or highest number = highest priority depending on convention). Can be preemptive (a higher priority process preempts current) or non-preemptive.

Problem — Starvation: In a heavily loaded system, a low-priority process may wait forever because high-priority processes keep arriving.

Solution — Aging: Gradually increase the priority of a process as it waits. For example, increase priority by 1 every 15 minutes. Eventually, even the lowest priority process will get the highest priority and execute.

2.6 System Calls: fork(), exec(), wait(), ps — Detailed

These are the fundamental process management system calls in Unix/Linux:

fork():

fork() creates a new process (child) by duplicating the calling process (parent). After fork(): both parent and child continue execution from the next instruction after fork(). The child is an exact copy of the parent (code, data, heap, stack, open files). Return value: 0 in child process, child's PID in parent, -1 on error. The two processes are now independent — changes in one do not affect the other (Copy-on-Write optimization: memory is shared until one writes to it).

exec():

exec() replaces the current process's memory image with a new program. After exec(), the calling process no longer exists as it was — it becomes the new program. exec() does NOT create a new process; it transforms the

existing one. Usually called after fork() to make the child run a different program. Returns only on error (since the original program is replaced).

Complete fork-exec example:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();       // Create child process

    if (pid < 0) {            // fork() failed
        perror("fork failed");
        return 1;
    } else if (pid == 0) {    // This is the CHILD process
        printf("Child PID: %d, Parent PID: %d\n", getpid(), getppid());
        execl("/bin/ls", "ls", "-l", NULL);  // Replace with 'ls -l'
        // If exec() returns, it failed:
        perror("exec failed");
        return 1;
    } else {                  // This is the PARENT process
        printf("Parent waiting for child %d\n", pid);
        int status;
        wait(&status);        // Wait for child to finish
        printf("Child finished with status %d\n", WEXITSTATUS(status));
    }
    return 0;
}
```

UNIT 3: CONCURRENCY AND SYNCHRONIZATION (5 Hours)

3.1 Principles of Concurrency — Detailed

> Definition: Concurrency is the property of a system where multiple processes or threads execute in overlapping time periods. On a single CPU, this is achieved through interleaving (rapid context switching). On multi-core CPUs, true parallel execution occurs.

Concurrency is powerful (it allows more work to be done) but introduces complex problems because processes share resources. The key problems are:

1. Race Condition:

A race condition occurs when the outcome of program execution depends on the relative timing of events (specifically, the order in which processes access shared data). The program produces correct results sometimes (lucky timing) and incorrect results other times (unlucky timing). Race conditions are very hard to debug because they are non-deterministic and hard to reproduce.

```
Classic race condition example — bank account:

Shared variable: balance = 1000

Thread 1 (deposit 500):    Thread 2 (withdraw 200):
  temp1 = balance            temp2 = balance
  temp1 = temp1 + 500        temp2 = temp2 - 200
  balance = temp1            balance = temp2

Interleaved execution:
  Thread1 reads balance: temp1 = 1000
  Thread2 reads balance: temp2 = 1000   ← reads before Thread1 writes!
  Thread1 writes: balance = 1500
  Thread2 writes: balance = 800         ← overwrites Thread1's update!

Expected: balance = 1000 + 500 - 200 = 1300
Actual:   balance = 800  ← deposit of 500 is LOST!
```

3.2 Critical Section Problem — Detailed

A critical section is a section of code that accesses shared resources (shared variables, files, I/O devices). The critical section problem is to design a protocol that processes can use to cooperate safely.

Each process has a structure like:

```
do {
    ENTRY SECTION        ← request permission to enter CS
      CRITICAL SECTION   ← access shared resource (e.g., update balance)
    EXIT SECTION         ← signal completion of CS
      REMAINDER SECTION  ← rest of the process code
} while(true);
```

A correct solution must satisfy ALL THREE of:

- **Mutual Exclusion:** If process Pi is executing in its critical section, no other process can be executing in its critical section. (Safety property — prevents corruption)
- **Progress:** If no process is in its CS and some processes want to enter, then selection of next process to enter CS cannot be postponed indefinitely. Only processes not in their remainder section can participate in the decision. (Liveness — prevents deadlock at CS level)
- **Bounded Waiting:** After a process has made a request to enter CS, there must be a bound (limit) on the number of times other processes are allowed to enter CS before this process's request is granted. (Fairness — prevents starvation)

3.3 Mutual Exclusion — Hardware and Software Approaches

Peterson's Solution (Software, 2 Processes):

Peterson's solution uses two shared variables: flag[2] (indicates if a process wants to enter CS) and turn (indicates whose turn it is to enter). This elegant solution satisfies all three requirements without hardware support.

```
// Shared variables (initialized before any process runs):
int turn;             // whose turn to enter CS (0 or 1)
bool flag[2] = {false, false};  // flag[i]=true: Pi wants to enter CS

// Process Pi's structure (i=0, j=1 or i=1, j=0):
do {
    flag[i] = true;     // Step 1: Announce intention to enter CS
    turn = j;           // Step 2: Give the other process courtesy
    while (flag[j] && turn == j); // Step 3: Wait if other wants to enter AND it's their turn

    /* CRITICAL SECTION */

    flag[i] = false;    // Step 4: Exit CS, retract intention

    /* REMAINDER SECTION */
} while(true);

Why it works:
- Mutual Exclusion: Both cannot pass the while-loop simultaneously
  (if both flag[0]=flag[1]=true, turn can only be 0 or 1, blocking one)
- Progress: The waiting process will eventually enter (turn changes)
- Bounded Waiting: Each process waits at most once (1 CS entry by the other)
```

Semaphores (Detailed):

> Definition: A semaphore is an integer variable that is accessed only through two atomic (indivisible) operations: wait() [also called P, down, or acquire] and signal() [also called V, up, or release].

Invented by Dijkstra in 1965. The key property is atomicity — the operations cannot be interrupted halfway. The OS guarantees this using hardware instructions (test-and-set) or by disabling interrupts during the operation.

```
Busy-Wait Semaphore (Spinlock):
wait(S)   { while(S <= 0); S--; }   // loop until S > 0, then decrement
```

```
signal(S) { S++; }                  // increment

Blocking Semaphore (better - no busy waiting):
typedef struct {
    int value;
    struct process *list;  // queue of waiting processes
} semaphore;

wait(S):
    S.value--;
    if (S.value < 0) {
        add this process to S.list;
        block();   // suspend process (put in Blocked state)
    }

signal(S):
    S.value++;
    if (S.value <= 0) {
        remove process P from S.list;
        wakeup(P);  // move P to Ready state
    }
```

Binary Semaphore (value: 0 or 1) = Mutex lock. Counting Semaphore (value: 0 to N) = track N instances of a resource.

Usage for mutual exclusion:

```
Semaphore mutex = 1;  // initialize to 1 (unlocked)

Process P1:                Process P2:
  wait(mutex);               wait(mutex);
  // CRITICAL SECTION        // CRITICAL SECTION
  signal(mutex);             signal(mutex);

Usage for process ordering (P2's statement must execute after P1's):
  Semaphore synch = 0;
  P1: execute statement A; signal(synch);
  P2: wait(synch); execute statement B;
```

Monitors:

A monitor is a high-level synchronization construct (like a class in OOP) that packages shared data and procedures together with a built-in mutual exclusion guarantee. Only one process can be active (executing) inside the monitor at any time. The compiler automatically inserts the necessary lock/unlock code.

```
monitor BoundedBuffer {
    int buffer[N];         // shared data — protected by monitor
    int count = 0;
    condition notFull;     // condition variable: wait if buffer full
    condition notEmpty;    // condition variable: wait if buffer empty

    procedure produce(item) {
        if (count == N) notFull.wait();  // suspend if full
        buffer[...] = item;
```

```
        count++;
        notEmpty.signal();  // wake up a waiting consumer
    }
    procedure consume() {
        if (count == 0) notEmpty.wait();  // suspend if empty
        item = buffer[...];
        count--;
        notFull.signal();  // wake up a waiting producer
        return item;
    }
}
```

UNIT 4: INTER-PROCESS COMMUNICATION (6 Hours)

4.1 Producer-Consumer Problem — Detailed with Full Semaphore Solution

Problem Statement: Two processes share a fixed-size circular buffer of N slots. The producer generates data items and places them in the buffer. The consumer removes items from the buffer and processes them. The challenge: prevent the producer from adding to a full buffer and prevent the consumer from removing from an empty buffer, while allowing concurrent operation.

```
// Shared variables:
Semaphore mutex = 1;  // binary semaphore for mutual exclusion on buffer
Semaphore empty = N;  // counting semaphore: number of empty slots (N initially)
Semaphore full  = 0;  // counting semaphore: number of full slots (0 initially)
int buffer[N];         // the shared buffer
int in = 0;            // index where producer inserts next item
int out = 0;           // index where consumer removes next item

// PRODUCER PROCESS:           // CONSUMER PROCESS:
void producer() {              void consumer() {
  while(true) {                  while(true) {
    item = produce_item();         wait(full);     // wait if no items
    wait(empty);  // wait if full  wait(mutex);    // lock buffer
    wait(mutex);  // lock buffer   item = buffer[out];
    buffer[in] = item;             out = (out+1) % N;
    in = (in+1) % N;               signal(mutex);  // unlock
    signal(mutex);// unlock        signal(empty);  // one more empty slot
    signal(full); // one more full consume_item(item);
  }                              }
}                              }

Why 3 semaphores?
- mutex: ensures only one process modifies in/out/buffer at a time
- empty: blocks producer when buffer has 0 empty slots
- full:  blocks consumer when buffer has 0 full slots

CRITICAL: wait(empty)/wait(full) MUST come before wait(mutex)!
If reversed: deadlock (producer holds mutex, waits for empty; consumer
wants mutex to signal empty — circular wait!)
```

4.2 Readers-Writers Problem — Detailed

Problem: A shared database is accessed by multiple processes. Readers only read the database (no modification). Writers read AND write. Rule: Multiple readers can read simultaneously (since reading doesn't corrupt data). A writer needs exclusive access — no readers or other writers allowed while writing.

```
// First Readers-Writers Solution (Readers preference — writers may starve):

int readcount = 0;       // number of currently active readers
Semaphore mutex = 1;     // protects readcount
```

```
Semaphore wrt = 1;       // exclusive access for writers; also blocks
                         // all readers when a writer is writing

// WRITER PROCESS:            // READER PROCESS:
void writer() {               void reader() {
  wait(wrt);    // lock DB       wait(mutex);
  // perform write              readcount++;
  signal(wrt);  // unlock DB    if (readcount == 1)  // first reader?
}                                 wait(wrt);  // lock out writers
                                signal(mutex);

                                // --- READ THE DATA ---

                                wait(mutex);
                                readcount--;
                                if (readcount == 0)  // last reader?
                                  signal(wrt);  // allow writers
                                signal(mutex);
                              }

Problem: Writer starvation. If readers keep arriving, readcount never
reaches 0, so wrt is never signaled, and writers wait forever.
Solution: Second Readers-Writers (Writers preference) uses additional
semaphores to give writers priority.
```

4.3 Dining Philosophers Problem — Detailed

> Problem: Five philosophers sit at a circular table. Between each adjacent pair is one fork (chopstick). There are 5 philosophers and 5 forks total. To eat, a philosopher needs BOTH the fork to their left AND the fork to their right. When not eating, a philosopher thinks. A philosopher can only pick up one fork at a time.

This is a model for resource allocation problems where processes need multiple resources and can deadlock by each holding one and waiting for the other.

```
// Naive (DEADLOCK-PRONE) solution:
Semaphore fork[5] = {1,1,1,1,1};  // each fork initially available

void philosopher(int i) {
  while(true) {
    think();
    wait(fork[i]);          // pick up LEFT fork
    wait(fork[(i+1) % 5]); // pick up RIGHT fork
    eat();
    signal(fork[i]);         // put down left fork
    signal(fork[(i+1) % 5]);// put down right fork
  }
}
// DEADLOCK: All 5 philosophers pick up left fork simultaneously.
// Now all wait for right fork. Circular wait = DEADLOCK.

// Solution 1: Allow only 4 philosophers to try simultaneously
Semaphore room = 4;  // at most 4 philosophers at table
void philosopher(int i) {
```

```
  wait(room);            // enter table (blocks 5th philosopher)
  wait(fork[i]);
  wait(fork[(i+1)%5]);
  eat();
  signal(fork[(i+1)%5]);
  signal(fork[i]);
  signal(room);          // leave table
}

// Solution 2: Asymmetric (breaks circular wait)
// Even philosophers: pick left then right
// Odd philosophers:  pick right then left
// This prevents circular wait condition.
```

4.4 Message Passing — Detailed

Message passing is an IPC mechanism where processes communicate by sending and receiving messages through the OS, without any shared memory. The OS provides the message queue/channel and handles synchronization.

Direct Communication: Processes name each other explicitly.

```
send(P, message)    — send message to process P
receive(Q, message) — receive message from process Q
Properties: link established automatically, exactly one link per pair, link usually bidirectional
```

Indirect Communication (Mailboxes): Messages sent to/received from mailboxes (ports).

```
send(A, message)    — send to mailbox A
receive(A, message) — receive from mailbox A
Properties: link only if both share a mailbox, can have multiple links, each link may be uni or bidire
```

Synchronous vs Asynchronous:

- **Blocking send:** Sender waits until message is received.
- **Non-blocking send:** Sender sends and immediately continues.
- **Blocking receive:** Receiver blocks until a message is available.
- **Non-blocking receive:** Receiver gets message or null.

4.5 Pipes — Detailed

A pipe is one of the oldest and simplest IPC mechanisms in Unix. It creates a one-way communication channel (unidirectional) between two processes. Data written to the write-end of the pipe can be read from the read-end. Internally, a pipe is a fixed-size kernel buffer (typically 64KB on Linux).

```
// Creating and using a pipe in C:
#include <unistd.h>

int fd[2];    // fd[0] = read end, fd[1] = write end
pipe(fd);     // create the pipe

if (fork() == 0) {  // CHILD: producer
```

```
    close(fd[0]);               // child doesn't read
    write(fd[1], "Hello!", 6);  // write to pipe
    close(fd[1]);
} else {            // PARENT: consumer
    close(fd[1]);               // parent doesn't write
    char buf[100];
    read(fd[0], buf, 100);      // read from pipe
    printf("Received: %s\n", buf);
    close(fd[0]);
}

// Named Pipes (FIFOs) — between unrelated processes:
mkfifo("/tmp/mypipe", 0666);   // create named pipe on filesystem
// Process 1: fd = open("/tmp/mypipe", O_WRONLY); write(fd, data, len);
// Process 2: fd = open("/tmp/mypipe", O_RDONLY); read(fd, buf, len);
```

UNIT 5: DEADLOCK (3 Hours)

5.1 Four Necessary Conditions for Deadlock — Detailed

> Definition: Deadlock is a situation where a set of processes are permanently blocked, each waiting for a resource held by another process in the set. No process can proceed, and no external intervention can resolve it automatically.

Deadlock can only occur if ALL FOUR of the following conditions hold simultaneously. Eliminating even ONE condition prevents deadlock.

1. Mutual Exclusion:

At least one resource must be held in a non-shareable mode — only one process can use it at a time. If another process requests that resource, it must wait until the resource is released. Example: a printer can only print one job at a time. Shareable resources (like read-only files) cannot cause deadlock because multiple processes can use them simultaneously.

2. Hold and Wait:

A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes. Example: Process P1 holds the printer and is waiting for the scanner, while P2 holds the scanner and is waiting for the printer.

3. No Preemption:

Resources cannot be preempted — a resource can only be released voluntarily by the process holding it after that process has completed its task. You cannot forcefully take a resource away from a process (at least for some resource types). If you could, you'd simply take the resource and give it to the waiting process.

4. Circular Wait:

There must exist a set of processes {P0, P1, P2, ..., Pn} such that P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, ..., Pn-1 is waiting for a resource held by Pn, and Pn is waiting for a resource held by P0. This forms a cycle.

```
Deadlock example with all 4 conditions:

Resources: Printer (P), Scanner (S) — one each, non-shareable
Process P1: holds Printer, requests Scanner
Process P2: holds Scanner, requests Printer

Condition 1: Mutual Exclusion ✓ (Printer and Scanner are non-shareable)
Condition 2: Hold and Wait ✓ (P1 holds Printer AND waits for Scanner)
Condition 3: No Preemption ✓ (cannot take Printer from P1 forcefully)
Condition 4: Circular Wait ✓ (P1→waits for Scanner held by P2→waits for Printer held by P1)
→ DEADLOCK!
```

5.2 Banker's Algorithm — Detailed with Numerical Example

The Banker's Algorithm (by Dijkstra) is a deadlock avoidance algorithm. Before granting a resource request, the OS checks if the system would remain in a 'safe state' after the allocation. If yes, the request is granted; otherwise, the process must wait.

Safe State: A state is safe if there exists at least one safe sequence — an ordering of all processes such that each process can obtain the maximum resources it needs using currently available resources plus resources held by preceding processes. If no safe sequence exists, the state is unsafe (may lead to deadlock).

```
COMPLETE NUMERICAL EXAMPLE:
5 processes: P0-P4    3 resources: A(10), B(5), C(7)

          Max        Allocation       Need = Max - Allocation
       A  B  C      A  B  C            A  B  C
P0:    7  5  3      0  1  0            7  4  3
P1:    3  2  2      2  0  0            1  2  2
P2:    9  0  2      3  0  2            6  0  0
P3:    2  2  2      2  1  1            0  1  1
P4:    4  3  3      0  0  2            4  3  1

Total Allocated: A=7, B=2, C=5
Total Available: A=10-7=3, B=5-2=3, C=7-5=2  → Available = [3,3,2]

SAFETY ALGORITHM:
Work = [3,3,2]   Finish = [F,F,F,F,F]

Step 1: Find i where Finish[i]=F AND Need[i] <= Work
  P0: Need=[7,4,3] <= [3,3,2]? NO
  P1: Need=[1,2,2] <= [3,3,2]? YES → Run P1
  Work = [3,3,2]+[2,0,0] = [5,3,2]   Finish[1]=T

Step 2: Work=[5,3,2]
  P0: Need=[7,4,3] <= [5,3,2]? NO
  P2: Need=[6,0,0] <= [5,3,2]? NO
  P3: Need=[0,1,1] <= [5,3,2]? YES → Run P3
  Work = [5,3,2]+[2,1,1] = [7,4,3]   Finish[3]=T

Step 3: Work=[7,4,3]
  P0: Need=[7,4,3] <= [7,4,3]? YES → Run P0
  Work = [7,4,3]+[0,1,0] = [7,5,3]   Finish[0]=T

Step 4: Work=[7,5,3]
  P2: Need=[6,0,0] <= [7,5,3]? YES → Run P2
  Work = [7,5,3]+[3,0,2] = [10,5,5]  Finish[2]=T

Step 5: Work=[10,5,5]
  P4: Need=[4,3,1] <= [10,5,5]? YES → Run P4
  Work = [10,5,5]+[0,0,2] = [10,5,7] Finish[4]=T

All Finish[i] = True → SAFE STATE!
Safe Sequence: <P1, P3, P0, P2, P4>
```

UNIT 6: MEMORY MANAGEMENT (7 Hours) — VERY IMPORTANT ★★★

6.1 Memory Management — Complete Overview

Memory management is one of the most complex and important functions of the OS. The goals are: keep multiple processes in memory simultaneously (multiprogramming), protect each process from others, allow sharing where needed, and maximize memory utilization.

6.2 Paging — Detailed Explanation

> Definition: Paging is a memory management scheme that eliminates external fragmentation by dividing physical memory into fixed-size blocks called frames and dividing logical memory into same-size blocks called pages. Any page can be placed in any free frame — they need not be contiguous.

Key concepts:

- **Page:** Fixed-size block of logical (virtual) memory. Typical sizes: 4KB, 8KB, or 2MB (for huge pages in Linux).
- **Frame:** Fixed-size block of physical memory. Same size as a page.
- **Page Table:** Per-process data structure maintained by OS that maps page numbers to frame numbers.
- **Page Table Base Register (PTBR):** CPU register that points to the current process's page table.
- **Translation Lookaside Buffer (TLB):** Hardware cache that stores recent page→frame translations for fast lookup.

```
ADDRESS TRANSLATION IN PAGING:

Logical Address (generated by CPU):
  ┌──────────────────────────────────────────┐
  │   Page Number (p) │  Offset (d)    │
  └──────────────────────────────────────────┘
       bits [n-1 .. m]      bits [m-1 .. 0]

Physical Address:
  ┌──────────────────────────────────────────┐
  │  Frame Number (f) │  Offset (d)    │
  └──────────────────────────────────────────┘

Translation:
  1. Extract page number p from logical address
  2. Look up page table: f = page_table[p]
  3. Physical address = f * page_size + offset d

DETAILED EXAMPLE:
  Page size = 4KB = 4096 bytes
  Logical address = 13056
  Page number  = 13056 / 4096 = 3  (integer division)
  Offset       = 13056 % 4096 = 768
```

```
Page table[3] = 6  (page 3 is in frame 6)
Physical address = 6 * 4096 + 768 = 24576 + 768 = 25344

Address space: 32-bit → 4GB virtual, page size 4KB
Number of pages = 4GB / 4KB = 1M pages
Page table size = 1M entries × 4 bytes = 4MB per process!
```

6.3 Page Replacement Algorithms — Detailed with Complete Examples

A) FIFO (First In First Out) — Detailed

FIFO replaces the page that has been in memory the longest. Easy to implement using a queue — when a page fault occurs and all frames are full, evict the page at the front of the queue (oldest) and add the new page to the back.

Belady's Anomaly: In FIFO, increasing the number of frames can sometimes INCREASE the number of page faults. This is counterintuitive and makes FIFO less reliable.

```
Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
Frames = 3:
Ref:  1   2   3   4   1   2   5   1   2   3   4   5
F1:   1   1   1   4   4   4   5   5   5   5   4   4
F2:   -   2   2   2   1   1   1   1   1   3   3   5
F3:   -   -   3   3   3   2   2   2   2   2   2   2
Flt:  F   F   F   F   F   F   F   -   -   F   F   F   = 9 faults

Same string with Frames = 4 (Belady's Anomaly):
Ref:  1   2   3   4   1   2   5   1   2   3   4   5
F1:   1   1   1   1   1   1   5   5   5   5   4   4
F2:   -   2   2   2   2   2   2   1   1   1   1   5
F3:   -   -   3   3   3   3   3   3   2   2   2   2
F4:   -   -   -   4   4   4   4   4   4   3   3   3
Flt:  F   F   F   F   -   -   F   F   F   F   F   F   = 10 faults!
More frames → MORE faults! This is Belady's Anomaly.
```

B) LRU (Least Recently Used) — Detailed

LRU replaces the page that has not been used for the longest time in the past. Based on the principle of temporal locality: recently used pages are likely to be used again soon. LRU does NOT suffer from Belady's anomaly.

Implementation: Stack-based (move page to top on each access; replace from bottom) or Counter-based (timestamp each access; replace page with oldest timestamp). Both are expensive in hardware, so approximations are used in practice.

```
Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
Frames = 3:

Step-by-step (tracking LRU order):
t=1: ref=7. Fault. Frames: [7]          LRU order: 7
t=2: ref=0. Fault. Frames: [7,0]        LRU order: 7,0
t=3: ref=1. Fault. Frames: [7,0,1]      LRU order: 7,0,1
t=4: ref=2. Fault. Replace LRU=7. Frames:[2,0,1]  LRU:0,1,2
```

```
t=5: ref=0. Hit!   Frames: [2,0,1]      LRU order: 1,2,0
t=6: ref=3. Fault. Replace LRU=1. Frames:[2,0,3]  LRU:2,0,3
t=7: ref=0. Hit!   Frames: [2,0,3]      LRU order: 2,3,0
t=8: ref=4. Fault. Replace LRU=2. Frames:[4,0,3]  LRU:3,0,4
t=9: ref=2. Fault. Replace LRU=3. Frames:[4,0,2]  LRU:0,4,2
t=10:ref=3. Fault. Replace LRU=0. Frames:[4,3,2]  LRU:4,2,3
t=11:ref=0. Fault. Replace LRU=4. Frames:[0,3,2]  LRU:2,3,0
t=12:ref=3. Hit!   Frames: [0,3,2]      LRU order: 2,0,3
t=13:ref=2. Hit!   Frames: [0,3,2]      LRU order: 0,3,2

Total Page Faults = 8  (vs FIFO=9, Optimal=5)
```

C) Optimal Page Replacement — Detailed

Replace the page that will not be used for the longest time in the future. This requires knowledge of the future reference string, which is impossible in real systems. However, it serves as a benchmark to measure how well other algorithms perform. It provably produces the minimum number of page faults for any given reference string and frame count.

```
Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 | Frames=3

t=1: ref=7. Fault. [7]
t=2: ref=0. Fault. [7,0]
t=3: ref=1. Fault. [7,0,1]
t=4: ref=2. Fault. Full. Next use of 7=never, 0=t5, 1=never.
            Replace 7 (or 1, whichever used furthest). Replace 1.
            Frames: [7,0,2]
t=5: ref=0. Hit. [7,0,2]
t=6: ref=3. Fault. Next use of 7=never, 0=t7, 2=t9. Replace 7.
            Frames: [3,0,2]
t=7: ref=0. Hit. [3,0,2]
t=8: ref=4. Fault. Next use of 3=t10, 0=t11, 2=t13. Replace 0.
            Wait — let's use next occurrences: 3→t10, 0→t11, 2→t13.
            Replace 2 (furthest). Frames: [3,0,4]
t=9: ref=2. Fault. Next:3→t10,0→t11,4→never. Replace 4. [3,0,2]
t=10:ref=3. Hit. [3,0,2]
t=11:ref=0. Hit. [3,0,2]
t=12:ref=3. Hit. [3,0,2]
t=13:ref=2. Hit. [3,0,2]

Total Page Faults = 5  ← MINIMUM POSSIBLE for this reference string
```

6.4 Thrashing — Detailed Explanation

> Definition: Thrashing is a condition where a process spends more time handling page faults (swapping pages in and out of disk) than actually executing productive instructions. CPU utilization drops dramatically even as the OS tries to increase it.

How Thrashing Develops: (1) Low CPU utilization → OS multiprogramming degree ↑ (adds more processes). (2) More processes → each process gets fewer frames. (3) Fewer frames → more page faults. (4) More page faults → processes spend more time waiting for disk I/O. (5) CPU utilization drops further. (6) OS sees low CPU utilization and adds even MORE processes → worse thrashing → vicious cycle.

Thrashing curve:



Prevention strategies:
1. Working Set Model: Track the 'working set' of each process (pages referenced in the last Δ time units). Allocate enough frames to hold each process's working set. If sum of working sets > total frames, suspend a process.

2. Page Fault Frequency (PFF) Control:
   - If a process's page fault rate > upper threshold → give it more frames
   - If fault rate < lower threshold → take frames away (give to others)
   - If fault rate high and no frames to give → suspend the process

UNIT 7: I/O MANAGEMENT AND DISK SCHEDULING (5 Hours)

7.1 Disk Scheduling — Detailed with Full Calculations

Disk access time has three components: (1) **Seek time** — time to move the read/write head to the correct cylinder (track). (2) **Rotational latency** — time for the disk to rotate the correct sector under the head. (3) **Transfer time** — time to transfer data. Seek time dominates (typically 5-10ms), so minimizing total head movement across cylinders is the primary goal of disk scheduling.

Given: Disk cylinders 0-199. Current head position: 53. Request queue: 98, 183, 37, 122, 14, 124, 65, 67.

A) FCFS — Detailed Calculation

```
Service order: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
Movements:  |53-98|+|98-183|+|183-37|+|37-122|+|122-14|+|14-124|+|124-65|+|65-67|
          =    45 +     85 +     146+     85  +     108+     110+      59+    2
          = 640 cylinders total

Assessment: Fair (respects arrival order) but very inefficient.
Head moves back and forth wildly. High average seek time.
```

B) SSTF — Detailed Calculation

```
At head=53: Distances: 98(45),183(130),37(16),122(69),14(39),124(71),65(12),67(14)
  Closest: 65 (distance 12) → move to 65
At head=65: Remaining distances: 98(33),183(118),37(28),122(57),14(51),124(59),67(2)
  Closest: 67 (distance 2) → move to 67
At head=67: Distances: 98(31),183(116),37(30),122(55),14(53),124(57)
  Closest: 37 (30) and 98 (31) → 37 closer → move to 37
At head=37: Distances: 98(61),183(146),122(85),14(23),124(87)
  Closest: 14 (distance 23) → move to 14
At head=14: Distances: 98(84),183(169),122(108),124(110)
  Closest: 98 (distance 84) → move to 98
At head=98: Distances: 183(85),122(24),124(26)
  Closest: 122 (distance 24) → move to 122
At head=122: Distances: 183(61),124(2)
  Closest: 124 (distance 2) → move to 124
At head=124: Distances: 183(59)
  Only: 183 → move to 183

Total: 12+2+30+23+84+24+2+59 = 236 cylinders
Assessment: Much better than FCFS. But starvation possible for far cylinders.
```

C) SCAN (Elevator) — Detailed Calculation

```
Head starts at 53, moving toward higher cylinders.
Service requests in increasing order: 65, 67, 98, 122, 124, 183
Then head reaches end (199 — no requests, but head goes to edge)
Then reverses direction (toward 0):
Service remaining in decreasing order: 37, 14
```

```
Order: 53→65→67→98→122→124→183→199→37→14
Movement: 12+2+31+24+2+59+16+162+23 = 331 cylinders

Wait — let me recalculate properly:
53→65: 12
65→67: 2
67→98: 31
98→122: 24
122→124: 2
124→183: 59
183→199: 16  (go to disk end)
199→37:  162 (reverse, pass 67,65,53 — already done)
37→14:   23
Total = 12+2+31+24+2+59+16+162+23 = 331 cylinders

Note: Different textbooks show 208 if head doesn't go to 199 but stops at
highest requested cylinder (183) before reversing. Both interpretations exist.
```

D) C-SCAN — Detailed Calculation

```
Head moves in one direction only (say, increasing).
Service: 65, 67, 98, 122, 124, 183
Then jump from 199 back to 0 (no service during jump)
Then service remaining: 14, 37

Order: 53→65→67→98→122→124→183→199→0→14→37
53→65:   12
65→67:   2
67→98:   31
98→122:  24
122→124: 2
124→183: 59
183→199: 16
199→0:   199  (jump — counted but no service)
0→14:    14
14→37:   23
Total = 12+2+31+24+2+59+16+199+14+23 = 382 cylinders

Assessment: C-SCAN provides more uniform waiting time than SCAN.
No process at one end waits longer than the other.
```

7.2 I/O Buffering — Detailed

Buffering is a technique to handle speed mismatches between producer and consumer (e.g., CPU produces data much faster than a printer can print). Without buffering, the CPU must wait for the slow device — extremely wasteful.

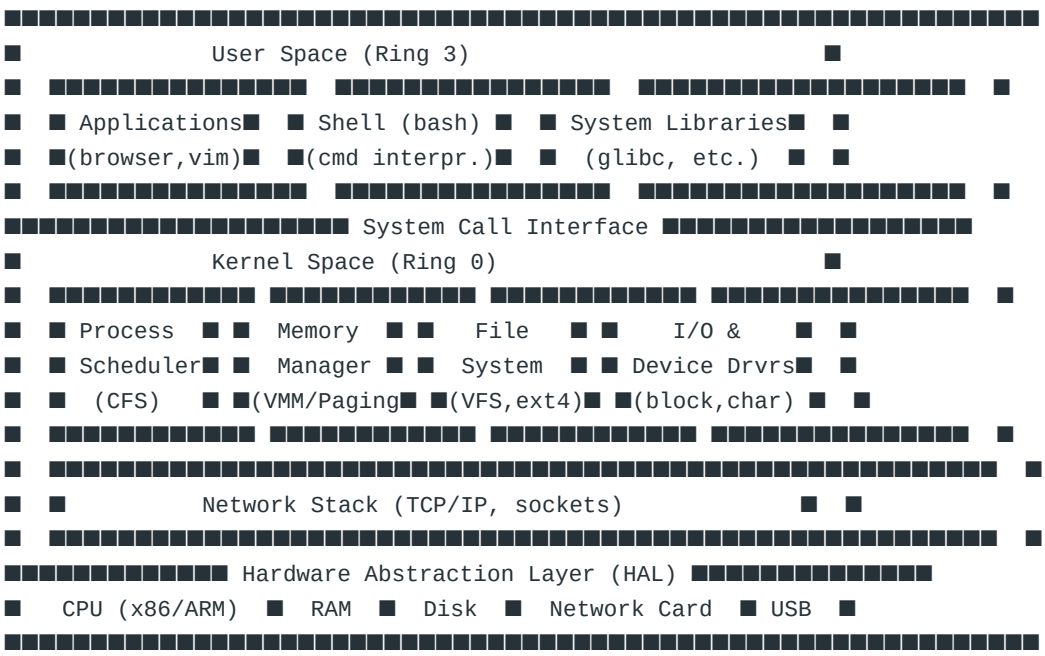
- **No Buffer:** Producer must wait for consumer to be ready. Very slow. CPU utilization drops to device speed.
- **Single Buffer:** OS provides a buffer in kernel memory. Producer fills buffer; consumer drains it. Overlap possible but limited — producer must wait if buffer is full.

- **Double Buffer (Ping-Pong):** Two buffers. Producer fills buffer A while consumer drains buffer B. Then switch. This allows full overlap of producer and consumer activity.
- **Circular Buffer:** Multiple buffers in a ring. Producer advances write pointer; consumer advances read pointer. Best for bursty I/O (like network packets).

UNIT 8: UNIX/LINUX OPERATING SYSTEM (5 Hours)

8.1 Architecture of Unix/Linux — Detailed

Linux is a monolithic kernel OS (all OS services run in kernel space). The architecture follows a layered model but unlike a strict layered OS, all kernel components can call each other directly for performance.



8.2 Linux Kernel — CFS Scheduler (Completely Fair Scheduler)

Linux uses the Completely Fair Scheduler (CFS) introduced in kernel 2.6.23. CFS aims to give every process a perfectly equal share of CPU time. Instead of fixed time quanta, CFS tracks virtual runtime (vruntime) — the amount of CPU time a process has consumed, weighted by its priority. The process with the lowest vruntime always runs next. Higher priority processes have their vruntime increase more slowly (they accumulate less virtual time per real second), so they run more often.

CFS uses a red-black tree (self-balancing BST) to store runnable processes, ordered by vruntime. Picking the next process is O(log n). This gives both efficiency and fairness.

8.3 Linux File System — Detailed

Linux uses the Virtual File System (VFS) abstraction layer, which provides a uniform interface for all file system types. Programs use the same open(), read(), write() calls regardless of whether the file is on ext4, NFS, /proc, or a USB drive.

```
VFS Architecture:
User program: open("/home/user/file.txt")
      ↓
VFS Layer: resolves path, finds inode, calls fs-specific ops
```

```
      ↓
ext4 driver: performs actual disk operations
      ↓
Block device layer: sends I/O requests to disk driver
      ↓
Disk hardware: reads/writes physical sectors

Key VFS data structures:
- inode: metadata for a file (size, permissions, timestamps, data block pointers)
- dentry: directory entry (maps file name to inode)
- file:   per-process open file (current file position, access mode)
```

8.4 Important Linux/Unix Commands — Detailed

```
PROCESS MANAGEMENT:
  ps aux          List all running processes with CPU/mem usage
  ps -ef          Full format process listing with PPID
  top             Real-time dynamic process monitor
  htop            Enhanced top with colors, mouse support
  kill -9 PID     Forcefully terminate process (SIGKILL)
  kill -15 PID    Graceful terminate (SIGTERM — default)
  killall nginx   Kill all processes named 'nginx'
  nice -n 10 cmd  Run command with lower priority (+10 niceness)
  renice -5 PID   Increase priority of running process
  bg / fg         Move job to background/foreground
  jobs            List background jobs

FILE SYSTEM:
  ls -la          Long listing with hidden files
  chmod 755 file  Set permissions: rwxr-xr-x
  chown user:grp  Change ownership
  df -h           Disk free space (human-readable)
  du -sh /var     Directory size
  find / -name x  Find file named 'x'
  mount /dev/sdb1 /mnt  Mount filesystem
  fsck /dev/sda1  Check and repair filesystem

NETWORK:
  ifconfig eth0   Show/configure network interface
  ip addr show    Modern replacement for ifconfig
  netstat -tulpn  Show listening ports
  ss -tulpn       Modern replacement for netstat
  ping google.com Test connectivity
  traceroute x    Show network path to host
  iptables -L     Show firewall rules
  ssh user@host   Secure shell remote login
```

UNIT 9: VIRTUALIZATION CONCEPTS

9.1 Virtualization — Detailed Explanation

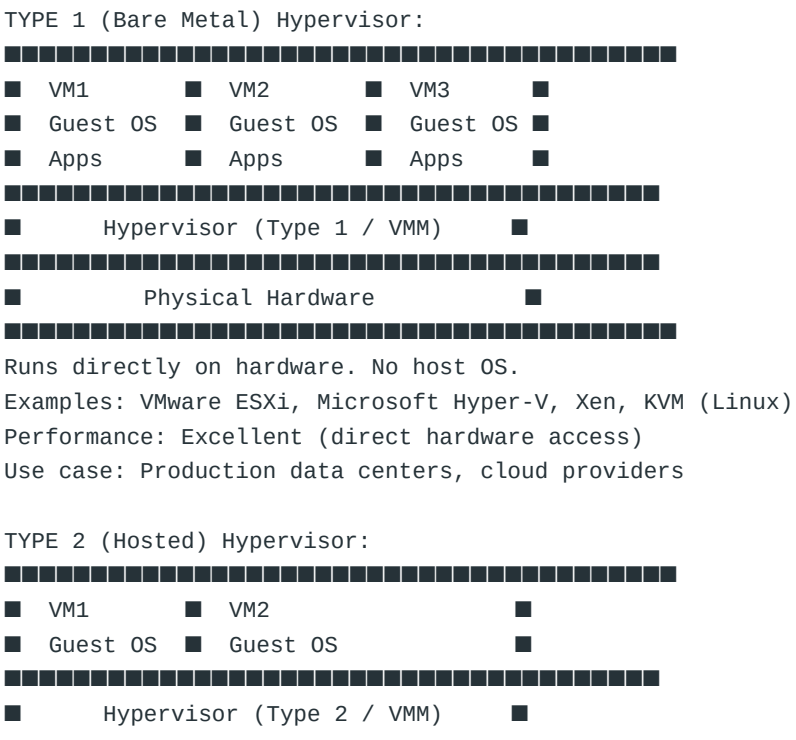
> Definition: Virtualization is the technology that creates a virtual (software-based) version of something — a server, storage device, network, or operating system — where the framework divides the resource into one or more execution environments. Each virtual environment is isolated, believing it has dedicated hardware.

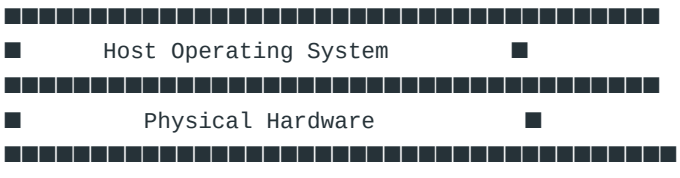
Virtualization was first developed by IBM in the late 1960s for mainframes. It was reinvented for x86 hardware in the early 2000s by VMware and has since become the foundation of cloud computing (AWS, Azure, GCP all use virtualization).

Benefits of Virtualization:

- **Server Consolidation:** Run multiple VMs on one physical server instead of one OS per server. Reduces hardware cost, power consumption, and data center space.
- **Isolation:** Each VM is fully isolated. A crash or security breach in one VM does not affect others.
- **Snapshots and Cloning:** Can take a snapshot of VM state and restore it instantly. Can clone a VM for testing or load balancing.
- **Live Migration:** Move a running VM from one physical host to another without downtime. Used for load balancing and hardware maintenance.
- **Development and Testing:** Developers can test software on multiple OS configurations on a single laptop.

9.2 Types of Hypervisors — Detailed






```
Runs on top of a host OS. Easy to install.
Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop
Performance: Slightly lower (extra OS layer)
Use case: Development, testing, running multiple OS on desktop
```

9.3 Paravirtualization vs Full Virtualization

Feature	Full Virtualization	Paravirtualization
Guest OS	Unmodified (runs as-is)	Modified (aware of virtualization)
Mechanism	Hypervisor emulates all hardware	Guest uses hypercalls to hypervisor
Performance	Slightly lower (emulation overhead)	Better (direct hypercalls, no emulation)
Compatibility	Any OS (Windows, Linux, etc.)	Only modified OSes (patched Linux kernels)
Examples	VMware ESXi, VirtualBox	Xen with PV kernel, Hyper-V with drivers

9.4 JVM and Android ART — Detailed

Java Virtual Machine (JVM):

The JVM is a process virtual machine that executes Java bytecode. Unlike OS hypervisors (which virtualize hardware), the JVM virtualizes a computing environment for a programming language. Java source code is compiled to bytecode (.class files) by the Java compiler. The JVM interprets this bytecode or compiles it to native machine code via Just-In-Time (JIT) compilation at runtime.

Write Once Run Anywhere (WORA): Java bytecode is the same regardless of the host OS. Different JVM implementations exist for Windows, Linux, macOS, etc. The same .jar file runs on all platforms.

Android Runtime (ART):

ART is the application runtime environment used by Android (since Android 5.0 Lollipop, replacing Dalvik). ART compiles Android application bytecode (DEX format — Dalvik Executable) to native machine code at **install time** using Ahead-of-Time (AOT) compilation. This means the app starts faster and runs more efficiently (no JIT overhead at runtime), but installation takes longer and uses more storage.

```
Comparison: JVM vs ART:

JVM (Java SE):                  Android ART:
  Source: .java file              Source: .java/.kt file
  Compiled to: .class (bytecode)  Compiled to: .dex (Dalvik bytecode)
  Execution: JIT compilation      Execution: AOT → native .oat files
  When compiled: at runtime       When compiled: at app install time
  Storage: small .jar files       Storage: larger (pre-compiled native)
  Startup: slightly slow (JIT)    Startup: fast (already native)
  Runtime: fast (JIT optimizes)   Runtime: very fast (already native)
  GC: multiple GC algorithms      GC: Concurrent Copying GC (low pause)
```

GTU EXAM PREPARATION — HIGH PRIORITY TOPICS

Most Frequently Asked Questions in GTU OS Exams:

Unit 6 — Memory Management (Most Important):

- Explain paging with diagram. Derive physical address from logical address. (4-7 marks)
- Solve page replacement algorithm (FIFO/LRU/Optimal) for a given reference string. (7 marks)
- Explain thrashing. What is its cause and how can it be prevented? (4 marks)
- Explain demand paging and page fault handling steps. (4-5 marks)
- Compare fixed vs variable partitioning with fragmentation types. (4 marks)

Unit 2 — CPU Scheduling (Very Important):

- Calculate Avg Waiting Time and Avg Turnaround Time for FCFS/SJF/RR/Priority. Draw Gantt chart. (7 marks)
- Compare preemptive and non-preemptive scheduling. (3 marks)
- Explain Round Robin scheduling with its advantages and disadvantages. (4 marks)

Unit 5 — Deadlock (Important):

- Solve Banker's Algorithm to determine if system is in safe state. Find safe sequence. (7 marks)
- Explain four necessary conditions for deadlock with examples. (4 marks)
- Explain deadlock prevention, avoidance, detection, and recovery. (7 marks)

Unit 4 — IPC and Synchronization (Important):

- Explain Producer-Consumer problem with semaphore solution. (5-7 marks)
- Solve Dining Philosophers problem. Explain deadlock-free solutions. (5 marks)
- Explain Peterson's solution for critical section. Prove mutual exclusion. (4 marks)

Unit 7 — Disk Scheduling (Important):

- Calculate total head movement for FCFS/SSTF/SCAN/C-SCAN for a given request queue. (7 marks)
- Compare disk scheduling algorithms. Which is best and why? (4 marks)

Key Formulas to Remember:

```
Turnaround Time (TAT) = Completion Time - Arrival Time
Waiting Time (WT)     = TAT - Burst Time
Response Time         = Time of First Response - Arrival Time
Avg Waiting Time      = Sum of all WT / Number of Processes

Paging:
  Page Number = Logical Address / Page Size   (integer division)
  Offset      = Logical Address % Page Size   (modulo)
  Physical Address = Frame Number * Page Size + Offset
```

```
  Number of Pages = Logical Address Space / Page Size
  Page Table Size = Number of Pages * Entry Size (typically 4 bytes)

Multiprogramming CPU Utilization:
  CPU Utilization = 1 - p^n   (p = I/O wait fraction, n = # processes)
```

Reference Books:

- Abraham Silberschatz, Peter Galvin, Greg Gagne — *Operating System Concepts* (Dinosaur Book) — Most recommended for GTU
- Andrew Tanenbaum — *Modern Operating Systems* — Excellent theoretical coverage
- William Stallings — *Operating Systems: Internals and Design Principles*
- D.M. Dhamdhere — *Operating Systems: A Concept-Based Approach* — Good for GTU pattern

All the Best for Your GTU Examinations!